

Days 13-16

Serving under an SLO, the Week 2 consolidation day, and quantization in two parts. Continues the Days 1-4, 5-8 and 9-12 notes. Same model and card throughout: Qwen2.5-1.5B, fp16, one RTX 3060 12GB, with an RTX 3090 used once as a control.

Day	Topic	The one thing you must be able to derive
13	Serving under an SLO	Why the knee moved with neither card, and what that proves
14	Week 2 consolidation	All four anchor answers, cold, timed, out loud
15	Quantization: the argument	Why weight-only quantization is a bandwidth play, in bytes
16	Quantization: the methods	Why your own 4-bit run got 2.27x slower, mechanically

Where these four days sit. Days 13 and 14 close Week 2 by turning the batching sweep into serving vocabulary and then rehearsing everything measured so far. Days 15 and 16 open Week 3 on the strongest differentiator available: a quantization result that went the *wrong* way and that can be explained exactly. A result that contradicts the naive story, understood, is worth more in an interview than three results that confirm it.

Day 13 · Serving under an SLO: the Pareto, goodput, and preemption

Day 12 established the two ceilings. Day 13 is the question an interviewer actually asks: given those curves, **what batch size do you ship?** The answer is never "the one with the highest throughput", and being able to say why is the whole day.

13.1 Throughput and latency are one curve, not two

Every row of the sweep is a point on a Pareto frontier: decode throughput cannot be improved without paying per-token latency. Plotting one against the other collapses the whole table into the shape that matters.

EXAMPLE 13-A: THE PARETO FRONTIER, RTX 3060, REALISTIC POOL

batch	TPOT ms	decode tok/s	tok/s per ms of TPOT
1	11.28	88.6	7.9
16	17.18	931.3	54.2
32	21.35	1498.6	70.2
48	23.25	2064.4	88.8
64	23.73	2696.5	113.6
96	36.81	2608.0	70.9
128	44.46	2879.0	64.8
192	72.26	2657.1	36.8
256	86.29	2966.7	34.4

the last column is throughput bought per unit of latency spent. it rises to batch 64 and falls monotonically after. that is the knee stated as a ratio, rather than eyeballed off a plot.

Note batch 96: throughput actually *fell* against batch 64 while TPOT rose 55%. That point is strictly dominated, meaning there is no SLO for which it is the right choice. A frontier with dominated points on it is normal and worth pointing out, because it shows the measurement had enough resolution to see non-monotonicity rather than smoothing it away.

13.2 Choosing a batch size, in the order you would actually do it

1. Write the SLO first, as a TPOT budget and a TTFT budget. "p95 TPOT under 25 ms" is an SLO; "as fast as possible" is not.
2. Find the largest batch that still meets the TPOT budget. At 25 ms on this card that is batch 64, at 23.73 ms, with 1.27 ms of margin.
3. Check the TTFT budget separately, because it is the constraint that usually binds first.

4. Check *which* ceiling is being hit, because the fix differs: more memory for an admission ceiling, a bigger or better-utilised card for a compute ceiling.

The TTFT trap. Step 2 alone gives batch 64. But TTFT at batch 64 is **5.516 s**, against 0.112 s at batch 1: a 49x regression. If the SLO says "first token in under 2 seconds", the entire TPOT analysis is irrelevant and the answer is batch 16 (TTFT 1.404 s, decode 931 tok/s), which is **2.9x less throughput** than the knee. Two SLOs, one curve, batch sizes a factor of four apart. This is the answer that separates someone who has run a sweep from someone who has read about one.

13.3 Goodput is the metric that stops you gaming throughput

Throughput counts every token emitted. Goodput counts only tokens belonging to requests that met their SLO. The distinction exists because throughput is trivially maximised by a policy that ruins latency for everyone, and the number still goes up.

EXAMPLE 13-B: THROUGHPUT UP 10%, GOODPUT TO ZERO

SLO: TPOT <= 25 ms.

```
batch 64: TPOT 23.73 ms  decode 2696.5 tok/s  -> meets SLO
          goodput = 2696.5 tok/s  (every token counts)

batch 256: TPOT 86.29 ms  decode 2966.7 tok/s  -> violates SLO by 3.5x
          throughput = 2966.7 tok/s  (+10%, and the dashboard is greener)
          goodput     = 0 tok/s      (not one request met its budget)
```

the operator who optimises the first number ships batch 256, and every user sees a 3.5x slower stream. the goodput column is what makes that visible.

13.4 Preemption: what a paged engine does when the pool runs out

The constrained-pool run from Day 12 was a scheduler experiment as much as a memory one. With a pool holding 25 concurrent sequences, batch 32 does not fail and does not thrash. It queues.

constrained pool: 1224 blocks x 16 tokens = 19,584 tokens = 25 sequences

batch	TPOT ms	decode tok/s	queued	
16	17.07	937.3	False	
32	33.18	964.5	TRUE	<- flips at the first batch above 25
48	43.92	1093.0	TRUE	
64	67.38	949.8	TRUE	

Two mechanisms are available to a paged engine when admission fails, and they are worth naming separately:

- **Swapping.** Copy a victim sequence's KV blocks out to host memory, and bring them back when it is rescheduled. Costs PCIe bandwidth twice, and preserves the computed KV.
- **Recomputation.** Drop the victim's blocks entirely and re-run its prefill on resume. Costs compute, needs no transfer. Cheaper than swapping whenever prefill is fast relative to the link, which on this card it is: prefill runs at **4205 tok/s**.

Both are why the constrained curve degrades gracefully to about 1000 tok/s rather than collapsing. Contrast the HuggingFace baseline, which under the same memory pressure does not queue, does not swap, and does not recompute: it raises `OutOfMemoryError` and the process dies. That is the same contrast as the OOM experiment, seen from the scheduler's side instead of the allocator's.

13.5 The cross-GPU control: is the knee a property of the model or of the card?

This is the most defensible part of the batching work, because it was a stated prediction that was then tested on different hardware.

The prediction: the knee is where the per-sequence marginal cost catches the shared weight read. The weight read is a property of the *model*, so the knee should sit at roughly the same batch size on a different card, even though every absolute number moves.

EXAMPLE 13-C: THE SAME SWEEP ON AN RTX 3090 (2.6X BANDWIDTH, 1.4X COMPUTE)

batch	3060 TPOT	3090 TPOT	3060 tok/s	3090 tok/s	ratio	
1	11.28	4.77	88.6	209.7	2.37x	
16	17.18	6.13	931.3	2608.8	2.80x	
32	21.35	6.60	1498.6	4845.3	3.23x	
64	23.73	7.87	2696.5	8127.5	3.01x	<- knee, both
96	36.81	11.19	2608.0	8582.6	3.29x	
128	44.46	13.16	2879.0	9728.3	3.38x	
256	86.29	25.32	2966.7	10112.1	3.41x	

TPOT is flat through batch 64 on BOTH cards and breaks at 96 on BOTH cards.
every throughput number scales by 2.4x to 3.4x. the knee does not move.

What the control proved, and what it refuted. Confirmed: the knee location is a model property. It stayed at 64 on a card with 2.6x the bandwidth. Refuted: the plateau prediction. I predicted 6000-7000 tok/s on the 3090 from the bandwidth ratio alone and measured **10,112**. Bandwidth was the wrong single axis, because the plateau is set by a skinny GEMM whose ceiling the 3090's extra compute also lifts. Say both halves. A prediction that half-failed and was diagnosed is a stronger signal than one that succeeded.

13.6 Why the plateau is not either roofline

The natural assumption is that a plateau means a hardware ceiling. Check it in numbers, on the 3060 at batch 256:

compute: ~9 TFLOPS achieved of ~51 TFLOPS peak = 16%
bandwidth: ~90 GB/s achieved of 360 GB/s peak = 25%

neither is saturated, and throughput still will not rise.

The reason is operation shape. Batched decode is a GEMM with $M = \text{batch}$, which stays skinny even at 256 against a K and N in the thousands. A skinny GEMM is occupancy-bound rather than FLOP-bound, and tops out well under tensor-core peak. Meanwhile the paged KV reads are scattered across blocks, so they do not reach streaming bandwidth either. The plateau is the ceiling of *those specific kernels*, not of the GPU.

The naive roofline agrees on the shape and fails on the number: it predicts an ideal batch of 142 at fp32-accumulate peak, or 280 at fp16-accumulate peak, against a measured 64. That 2x-to-4x gap is itself a finding, and it is the honest form of the roofline claim. The roofline tells you which regime you are in, not what number you will hit.

13.7 Self-test, Day 13

Q1. Your sweep says batch 256 is fastest. Do you ship it?

No. It is 10% more decode throughput than batch 64 for 3.6x the per-token latency, and against a 25 ms TPOT SLO its goodput is zero while batch 64's is 2696 tok/s. I would ship 64, and if TTFT is also budgeted I would ship lower still: TTFT at 64 is 5.5 s against 0.11 s at batch 1.

Q2. How do you know your knee is not an artifact of that one GPU?

I reran the sweep on an RTX 3090 with 2.6x the bandwidth and 1.4x the compute. TPOT stayed flat through batch 64 and broke at 96 on both cards, so the knee did not move, which is what the $T_{\text{fixed}} + c_1 \times \text{batch}$ model predicts, because the fixed term is the model's weight read. The plateau height did move, 2967 to 10,112 tok/s, and my bandwidth-only prediction of 6-7k was wrong for a reason I can name: the second card also lifts the skinny-GEMM ceiling.

Q3. Your throughput flattened. Which resource ran out?

On this card, none of them. At batch 256 I was at 16% of compute peak and 25% of bandwidth peak. The limit is kernel shape: decode is a skinny GEMM that is occupancy-bound, plus scattered paged KV reads. That is also why the knee is at 64 and not at the roofline's predicted 142.

Sources: Kwon et al., PagedAttention / vLLM (preemption by swap or recompute) · Yu et al., Orca (OSDI 22) · Zhong et al., DistServe (goodput as the objective) · Databricks, LLM Inference Performance Engineering

Day 14 · Week 2 consolidation: everything measured, said cold

This day reads nothing new. Week 2 covered the two topics an interviewer is most likely to go deep on, and the failure mode now is recognition without recall: every one of these ideas feels obvious on the page and disappears under an interruption. The work today is to produce all of it from a blank sheet, timed, out loud.

14.1 The blank-page drill, in order

Do these in one sitting, writing only, no notes open. Thirty to forty minutes total.

#	Produce from memory	Correct answer, to check against after
1	The KV-cache size formula and its value for this model	$2 \times 28 \times 2 \times 128 \times 2 = 28,672 \text{ B} = 28 \text{ KiB/token}$
2	The three wastes PagedAttention names	reservation, internal fragmentation, external fragmentation
3	The online-softmax recurrence, all three accumulators	m, l, O , with $c = \exp(m_{\text{old}} - m_{\text{new}})$
4	Why FlashAttention does nothing at batch-1 decode	score matrix is $1 \times n$, no quadratic term and no tile worth keeping in SRAM
5	What does help instead, and its combine step	flash-decoding / split-KV, merged with the same rescale rule
6	The step-time model and both of its limits	$T_{\text{fixed}} + c1 \times \text{batch}$; linear then asymptotic to $1/c1$
7	The two plateaus and the distinct cause of each	$\sim 2700 \text{ tok/s}$ compute ceiling; $\sim 1000 \text{ tok/s}$ KV-admission ceiling
8	The idle-versus-kernels split of the vLLM win	25.34 ms idle removed (90%), 2.60 ms faster kernels (9%)

14.2 The number sheet, which should be reproducible without the CSVs

MODEL	Qwen2.5-1.5B, fp16. 1,543,714,304 params. 28 layers, 12 query heads, 2 KV heads, head_dim 128, hidden 1536. weights resident 2945 MiB, 3.09 GB streamed per decode step.
CARD	RTX 3060 12GB, Ampere sm_86, 28 SMs. $\sim 51 \text{ TFLOPS}$ fp16 (fp32-accumulate), 360 GB/s spec bandwidth, 291.5 GB/s measured achievable read bandwidth. ridge point $51e12 / 360e9 = 142 \text{ FLOP/byte}$.
PHASES	prefill 4205 tok/s (122 ms for 512 tokens) decode 28.1 tok/s (HF) $\rightarrow$ 150x per-token gap
ENGINES	HF 28.1 $\rightarrow$ vLLM 79.9 tok/s at batch 1 = 3.2x of the 28.09 ms/token gap: 25.34 ms idle (90%), 2.60 ms kernels (9%) HF step: 1198 kernels, 1198 CPU launches, 62.3% device idle vLLM eager: 356 kernels (3.4x fewer) $\leftarrow$ fusion graphs on: same 356 kernels, 17 launches, idle ~ 0 $\leftarrow$ CUDA graphs vLLM decode runs at 85% of the 291.5 GB/s achievable bandwidth
MEMORY	OOM at 123,565 tokens of context analytical 28 KiB/token, measured slope 62 KiB/token (2.21x) at OOM: 10,437 MiB allocated, 11,764 MiB reserved, 1.3 GiB stranded decode decayed 30.03 $\rightarrow$ 2.93 tok/s across that sweep (10.3x)
BATCHING	88.6 $\rightarrow$ 2696 tok/s from batch 1 to 64 (30x), then 2967 at 256 TPOT 11.28 $\rightarrow$ 23.73 $\rightarrow$ 86.29 ms across the same range $T_{\text{fixed}} \sim 11 \text{ ms}$, $c1 \sim 0.20 \text{ ms/seq}$, $1/c1 \sim 3000 \text{ tok/s}$ 3090 control: knee still 64, plateau 10,112 tok/s

14.3 The four anchor answers, timed

Say each out loud against a timer. The target is 60 to 90 seconds. Over 90 means it is being recited from a memorised paragraph rather than reconstructed, and it will not survive an interruption. Under 45 usually means a mechanism has been dropped.

1. **FlashAttention.** Tiling, online softmax, exactness, IO not FLOPs, then the batch-1 caveat and split-KV. The one anchor with no repo number behind it, so the derivation has to carry it.

2. **The KV-cache OOM.** 28 KiB analytical against 62 KiB measured, the `torch.cat` double-copy that explains the 2.21x, then fragmentation at the crash: 1.3 GiB free and unusable.
3. **The batching knee.** 88.6 to 2696 tok/s, the `T_fixed + c1 x batch` derivation, the knee at 64, and the 3090 control.
4. **HF versus vLLM.** The 2x2 ablation, 90% idle and 9% kernels, and the non-additivity of compile and graphs.

The interruption drill. Have someone stop you mid-answer with "wait, why is it 2 KV heads and not 12?" or "why does that factor of 2 appear twice?" and resume. Interviews are not monologues. An answer that only works uninterrupted is a memorised answer, and the difference is visible from across a table.

14.4 The three sentences most likely to be challenged

What you said	The challenge	The prepared answer
"Decode is memory-bound."	"Then why is your GPU 62% idle rather than bandwidth-saturated?"	Both are true and they are different layers. The kernel is memory-bound; the step is launch-bound. HF's decode achieved only 15-20% MBU precisely because idle dominated. vLLM removed the idle and reached 85% of achievable bandwidth, at which point the memory-bound claim is what remains.
"vLLM is 3.2x faster."	"Your two files say 79.9 and 88.6 tok/s. Which is it?"	79.9 is the standalone baseline script, 88.6 is batch 1 of the sweep, run under a different pool configuration and a different vLLM build. I quote 3.2x from the baseline pair because that is the matched comparison; the sweep number is internally consistent with the rest of the sweep.
"68% of peak bandwidth."	"Which peak?"	Against the 360 GB/s spec figure. Against the 291.5 GB/s I measured with a read benchmark on the same card, it is 85%. I quote the measured ceiling, because the spec number is a number no kernel on this card reaches.

14.5 Exit bar for Week 2

1. Derive the online-softmax rescale on a whiteboard with no notes and verify it against a direct four-element computation.
2. State why FlashAttention does nothing for batch-1 decode, and what does instead.
3. Tell the OOM story in under 90 seconds with four numbers in it.
4. Name the two batching plateaus, their values, and the distinct mechanism behind each.
5. Give the idle-versus-kernel split of the vLLM win, and say why the two optimizations are not additive.

Day 15 · Quantization, part 1: the argument, in bytes

Quantization is a **differentiator** topic rather than a guaranteed one, and the goal for these two days is one paragraph of mechanism per method plus one measured result that is genuinely your own. Day 15 builds the argument from the roofline. Day 16 does the methods, and then the result that contradicts the argument.

15.1 Why quantization helps inference at all

Start where Day 5 ended. A batch-1 decode step reads all **3.09 GB** of fp16 weights to produce a single token, and does about $2 \times 1.54e9 = 3.1$ GFLOP of arithmetic doing it. That is an arithmetic intensity near 1 FLOP/byte, against a ridge point of 142. The step is nowhere near the compute ceiling and is entirely a memory-traffic problem.

So the lever is bytes. If the same computation can be driven from weights stored in 4 bits instead of 16, the dominant term in the step shrinks by 4x, and per-token latency should follow.

EXAMPLE 15-A: THE IDEAL BANDWIDTH FLOOR AT EACH WIDTH

weights = 1,543,714,304 params

fp16	2 bytes/param	= 3.087 GB	-> 3.087 / 291.5 GB/s	= 10.6 ms/token	(94 tok/s)
int8	1 byte /param	= 1.544 GB	-> 1.544 / 291.5	= 5.3 ms/token	(189 tok/s)
int4	0.5 byte/param	= 0.772 GB	-> 0.772 / 291.5	= 2.6 ms/token	(378 tok/s)

291.5 GB/s is the MEASURED achievable read bandwidth on this card, not the 360 GB/s spec figure. quoting the spec figure inflates every line above by 23%.

The sentence to have ready. "Weight-only quantization is a bandwidth play, not a compute play. At batch 1 the arithmetic intensity is about 1 against a ridge of 142, so I am paying for bytes moved and not for FLOPs done. Halving the bytes is the whole mechanism, and it follows that the win shrinks as batch size grows, because batching amortises the same weight read over more tokens and moves the step toward the compute-bound side."

15.2 The corollary that most people miss

If quantization wins by shrinking a *shared* read, then its benefit is largest exactly where that read is least amortised: batch 1. At batch 64, the same 3.09 GB is already being spread across 64 tokens, so the weight term is 64x less important per token and the step has moved toward the skinny-GEMM ceiling from Day 13. This is checkable against the measured numbers.

batch 1:	T_fixed ~ 11 ms	is essentially ALL of the 11.28 ms step
batch 64:	T_fixed ~ 11 ms	is 11 of a 23.73 ms step, so ~46%
		the other ~54% is c1 x 64 = 12.7 ms of per-sequence work
halving the weight bytes attacks T_fixed only:		
batch 1:	11.28 -> ~5.8 ms	(about 1.9x faster)
batch 64:	23.73 -> ~18.2 ms	(about 1.3x faster)

Same optimization, very different payoff, and the reason is a decomposition already measured. That is the kind of answer that shows a model rather than a memorised fact.

15.3 What actually gets quantized, and what does not

Tenant	Size here, fp16	Quantized?	Note
Weights	2945 MiB resident	Yes, the usual target	Static, shared across all sequences, read every step. Ideal target.
KV cache	28 KiB/token analytical	Sometimes, to int8 or fp8	Grows with context and batch. Quantizing it buys context length directly.
Activations	small at batch 1	Only in W8A8 schemes	Where the outlier problem lives. This is what SmoothQuant is about.
Attention scratch	~700 MiB at 120k ctx	No	Transient. Showed up in the OOM sweep as the residual above weights + 2 x KV.

15.4 The dequantization tax, stated before you meet it

A 4-bit weight is not something an Ampere tensor core can multiply. sm_86 has no int4 or fp8 tensor path that a general GEMM can use here, so the actual execution is:

```
for each weight tile:
  read 4-bit packed values from HBM          <- the saving, real
  look up / scale them back into fp16        <- the tax, per element
  feed fp16 tensor cores                     <- unchanged arithmetic
```

The read shrinks by 4x. The arithmetic does not shrink at all, and a dequantization step is added that did not exist before. Whether this is a win depends entirely on whether the step was actually bottlenecked on that read. Hold that thought: it is exactly what the measurement on Day 16 tests, and it is exactly where the measurement goes against the story.

15.5 Formats, precisely enough to be challenged on

Format	Bits	What it is
fp16	16	1 sign, 5 exponent, 10 mantissa. The baseline of record here.
bf16	16	1 sign, 8 exponent, 7 mantissa. Same range as fp32, less precision. Not used in this repo, deliberately, for comparability with V100 and T4 runs later.
int8	8	Uniform grid, needs a scale (and sometimes a zero point) per tensor, per channel, or per group. Symmetric per-channel is the common serving choice.
int4 / NF4	4	16 levels. NF4 places those levels at the quantiles of a normal distribution rather than uniformly, because trained weights are approximately Gaussian, so a uniform grid wastes most of its codes on the tails.
fp8 (e4m3/e5m2)	8	Hopper-class hardware feature. Nameable, not measurable on sm_86. Correct answer here is "aware of it, cannot measure it on my card".

Why NF4 rather than plain int4. Quantization error is minimised by putting your 16 available codes where the mass of the distribution is. Trained weights are roughly zero-mean Gaussian, so NF4 spaces its levels at the quantiles of a normal, making the codes information- theoretically optimal for that assumption. Combined with block-wise scaling (a separate scale per 64 weights, so one outlier cannot flatten a whole tensor) and double quantization (quantizing those scales too), that is the QLoRA recipe.

Day 16 · Quantization, part 2: the methods, and the run that got slower

16.1 The four methods, one paragraph of mechanism each

Method	The mechanism, in one sentence	What it is really solving
GPTQ	Quantize weights one column at a time and, after each one, update the remaining unquantized columns to compensate for the error just introduced, using second-order (Hessian) information from a small calibration set.	Rounding error is not independent across weights. Greedy per-weight rounding compounds; error-compensating rounding does not.
AWQ	Observe that a small fraction of weight channels (about 1%) matter far more for output quality, identify them from <i>activation</i> magnitudes rather than weight magnitudes, and scale those channels up before quantizing so they land on a finer part of the grid.	Not all weights are equally important, and importance is visible in the activations, not in the weights themselves.
SmoothQuant	Activations have huge outliers and weights do not, which makes W8A8 fail on the activation side. Migrate the difficulty: divide activations by a per-channel factor s and multiply the corresponding weight channels by s , leaving the product unchanged but making both sides quantizable.	An algebraic identity used to move dynamic range from the tensor that cannot take it to the one that can.
LLM.int8()	Past roughly 6.7B parameters, a few hidden dimensions develop systematic activation outliers that dominate the result. Decompose the matmul: run those few outlier dimensions in fp16 and everything else in int8, then add the two results.	Establishing that the outliers are a small, identifiable, structured set rather than noise, which is what makes mixed precision practical.

The organising observation across all four: **the hard part of quantization is not the rounding, it is the outliers**. GPTQ compensates for their error, AWQ protects them, SmoothQuant relocates them, LLM.int8() isolates them. Four answers to one problem is a much better framing than four unrelated acronyms.

16.2 KV-cache quantization, which is a different lever entirely

Weight quantization shrinks a fixed cost. KV quantization shrinks a *growing* one, which matters on exactly the axis the OOM experiment measured.

```
analytical KV at fp16      = 28 KiB/token
analytical KV at int8      = 14 KiB/token
```

```
at the measured slope of 62 KiB/token, weights+KV filled 12GB at 123,565 tokens.
halving only the KV term does not halve the slope, because the measured slope
includes HuggingFace's double-copy: peak is weights + 2 x KV, not weights + KV.
which is the useful point: a paged engine gets the full benefit, a torch.cat
loop gets the benefit twice over on a term that should never have doubled.
```

The interview version: KV quantization is the one quantization lever whose payoff scales with context length and batch size, so it is the one that matters for long-context serving, while weight quantization is the one that matters for batch-1 latency and for fitting the model at all.

16.3 The measured result: NF4 on this model and card

The same harness, the same 512-token prompt, the same 255 decode tokens, the same script. Only the dtype changed. Six fp16 runs and six NF4 runs.

EXAMPLE 16-A: NF4 AGAINST FP16, FROM RESULTS/BASELINE_HF.CSV

	fp16	NF4	change
weights resident	2945.29	1099.13 MiB	2.68x SMALLER
peak reserved	3360.00	1394.00 MiB	1966 MiB freed
prefill tok/s	4207	3652	13.2% slower
decode tok/s	28.16	12.42	2.27x SLOWER
decode ms/token	35.51	80.52	+45.0 ms per token

the memory result is exactly as predicted. the latency result is backwards, and by a lot.

The trap question, and it will be asked. "You just told me weight-only quantization is a bandwidth play. Your 4-bit run reads 2.68x fewer weight bytes. Why did it get 2.27x slower?" Anyone who has only read about quantization stalls here. The answer is in this repo's own profiling data.

16.4 The answer, in three steps

Step 1: the bandwidth win is real, and it is not the bottleneck. The fp16 HuggingFace decode step is **62.3% device idle** and fires 1198 kernels per token. It was never running at the bandwidth ceiling; it reached only 15-20% MBU. An optimization that reduces bytes read cannot help a step that is not waiting on bytes read. It is Amdahl's law applied to the wrong term.

```
fp16 decode, HF:  35.51 ms/token, of which ~62% is device idle
                  -> ~13.4 ms of actual device work
                  3.087 GB / 13.4 ms = 230 GB/s ... but that is the BUSY
                  window only. across the whole step it is 87 GB/s, ~24% of
                  the 360 GB/s spec figure. bytes were never the constraint.
```

Step 2: dequantization adds work to a step that was already launch-bound. Every quantized linear layer now runs a dequantize kernel before its GEMM. That is more kernels and more launches per token, on a loop where 62.3% of the time was already dispatch overhead. The optimization made the dominant term worse.

```
effective bandwidth achieved, whole step:
  fp16:  3.087 GB / 35.51 ms = 86.9 GB/s
  NF4:   1.152 GB / 80.52 ms = 14.3 GB/s

the NF4 run moves 2.68x fewer bytes in 2.27x more time. the extra
45.0 ms/token is not memory traffic at all. it is dequantization plus
the launches that carry it.
```

Step 3: the caveat that makes the answer correct rather than merely defensive. This measures *this* implementation on *this* card. bitsandbytes NF4 dequantizes to fp16 and calls an fp16 GEMM, and sm_86 has no int4 tensor-core path to skip that with. A serving stack with fused dequantize-and-matmul kernels, such as Marlin or an AWQ or GPTQ kernel in vLLM, folds the dequantization into the GEMM's prologue and does realise the bandwidth win. The result here is correct and specific, not a claim that 4-bit quantization is useless.

16.5 What the run *did* buy, quantified

The memory result is the real win and it converts directly into the other headline experiment.

```
weights freed = 2945.29 - 1099.13 = 1846.16 MiB

at the MEASURED OOM slope of 62 KiB/token:
  1846.16 MiB x 1024 / 62 KiB = 30,489 more tokens of context

at the ANALYTICAL 28 KiB/token (what a paged engine would actually see):
  1846.16 x 1024 / 28          = 67,511 more tokens of context

so on the same 12GB card, NF4 moves the OOM cliff from 123,565 tokens to
roughly 154,000 in the HF loop. it buys context, not speed.
```

The complete answer, as one paragraph. "Weight-only quantization is a bandwidth play, so I expected 4-bit to be faster at batch 1. It was 2.27x slower. The reason is that my baseline was not bandwidth-bound: profiling showed the HF decode step was 62.3% device idle across 1198 kernel launches, running at under a quarter of peak bandwidth. Shrinking the bytes attacks a term that was not binding, while dequantization adds kernels to the term that was. It is bitsandbytes-specific and card-specific, because sm_86 has no int4 tensor path, so the dequantize is a separate step rather than a fused GEMM prologue. What the run did buy is memory: 1846 MiB of weights freed, which is about 30,000 more tokens of context at my measured KV slope. Quantization bought me context, not latency, and the profiler explains exactly why."

16.6 Self-test, Days 15-16

Q1. Why does quantization help inference more than it helps training?

Inference at low batch is memory-bound with arithmetic intensity near 1 against a ridge of 142, so the cost is bytes moved and weights are the largest static tenant. Training is compute-bound, has a backward pass and optimizer state, and needs gradient precision. Quantization attacks the term that dominates inference and not the one that dominates training.

Q2. Would you expect a bigger or smaller win from quantization at batch 64 than at batch 1?

Smaller. The weight read is `T_fixed`, about 11 ms, and it is essentially the entire 11.28 ms step at batch 1 but only about 46% of the 23.73 ms step at batch 64, because `c1 × batch` has grown to 12.7 ms. Halving the weight bytes gives roughly 1.9x at batch 1 and roughly 1.3x at batch 64, on my own decomposition.

Q3. GPTQ and AWQ both do 4-bit weight-only. What is actually different?

GPTQ is error compensation: quantize column by column and update the not-yet-quantized columns with Hessian information so the accumulated error is corrected. AWQ is error avoidance: find the roughly 1% of channels that matter, identified from activation magnitudes rather than weight magnitudes, and scale them so they land on a finer part of the grid. One repairs the damage, the other steers around it.

Q4. Your 4-bit run got slower. Does that mean quantization does not work?

No, it means my baseline was not bandwidth-bound and my kernel was not fused. The step was 62.3% idle across 1198 launches, so cutting bytes could not help, and bitsandbytes NF4 dequantizes into a separate fp16 GEMM because `sm_86` has no int4 tensor path. A fused dequantize-and-matmul kernel such as Marlin on a stack that is actually bandwidth-bound is the configuration where the win appears. I measured one point precisely, and I can say exactly which of its assumptions to change.

Sources: Frantar et al., GPTQ · Lin et al., AWQ · Xiao et al., SmoothQuant · Dettmers et al., LLM.int8() · Dettmers et al., QLoRA (NF4) · Marlin, fused 4-bit GEMM kernels

Repo sources for every measured number above: `results/vllm_batch_sweep.csv` (both pool configurations and both GPUs), `results/baseline_hf.csv` (the fp16 and NF4 rows), `results/profile_summary.csv`, `results/oom_sweep.csv`, `docs/batching-results.md`, `docs/profiling.md`, `docs/oom-results.md`. Batch-sweep throughput is the `decode_throughput_tok_s` column, averaged over repeats. fp16-versus-NF4 decode figures are means over the six runs of each dtype. Where a CSV and a prose doc disagree, the CSV is used.